

Assignment 3: Fault-Tolerant Distributed Banking on Kubernetes

Due date: _____1405/08/29_____ Submission: ____ LMS,Github,Dockerhub _____

Assignment Reviewers: Alireza Bagherzadeh | Mehdi Shekari

Cloud-Native Deployment, Replication, Fault Tolerance & Disaster Recovery

1. Goal

Extend the Assignment 2 banking platform (sharding, replicas, shard-aware gateway, 2PC coordinator) and deploy it on Kubernetes (Minikube), keeping all of its functionality and correctness guarantees. Bonuses extend this to a multi-VM cluster and disaster recovery.

2. Core Requirements

- **Deployment:** Stateless and stateful workloads, multiple app replicas per shard, service discovery, an external entry point, routing that stays correct as Pods change; everything reproducible from manifests.
- **Databases:** Each shard has a persistent primary DB plus a working standby (real replication); data survives Pod deletion; documented recovery from primary failure with no inconsistent balances or duplicate effects.
- **Self-healing:** Failed Pods are recovered automatically, unhealthy replicas stop receiving traffic; application availability is distinguished from database availability.
- **Autoscaling:** Resource requests/limits and workload-driven autoscaling (not manual replica edits); correctness preserved while the replica count changes.
- **Updates:** Rolling update under load, rollback after a failed update, in-flight requests handled safely.
- **Config, security, observability:** ConfigMaps/Secrets, restricted communication (at least one enforced restriction, e.g. NetworkPolicy), logs and metrics for failures, availability and transaction states.
- **Transaction correctness:** All guarantees of Assignments 1 and 2 still hold during scaling, Pod failures, DB failover and updates. Temporary blocking is acceptable; unresolved operations must stay recoverable.

3. Failure Experiments (required, with details)

Provide repeatable experiments (commands, load generators, verification scripts). Correctness must be verified against actual database and transaction state, not only HTTP responses or Pod status.

#	Experiment	Expected Outcome
1	Application Pod Failure	Healthy replicas continue serving requests
2	Primary Database Failure	Database recovery preserves valid committed state
3	Concurrent Transactions	Account balances remain correct
4	Coordinator Interruption	Unresolved transactions are recovered correctly
5	Autoscaling Under Load	Scaling occurs without violating banking correctness
6	Rolling Update Under Load	Deployment changes preserve transaction integrity
7	Repeated Client Requests	No duplicate transaction effects

For each experiment, report the injected failure, the observed behavior, and the DB/transaction-state check (balances, money conservation, transaction status by ID) that proves correctness.

---

4. Advanced Bonus I: Multi-VM Kubernetes

- Three VMs: VM1 as the control plane, and VM2 and VM3 as worker nodes, forming a Kubernetes cluster.
- Application replicas spread across both workers; all components communicate across machines.
- Per shard: primary DB, a local standby/backup on the same worker, a second standby on the other worker with independent storage, and independent recovery backups.
- Interrupt one worker under concurrent load; recover with committed data intact and no two writable DBs.

5. Expert Bonus II: Permanent VM Loss & Disaster Recovery

- **Scenario A:** Worker destroyed permanently (VM and disks gone) during transactions: recover from surviving resources, keep acknowledged transactions, handle interrupted distributed transactions, restore DB redundancy.
- **Scenario B:** Control-plane VM destroyed permanently: restore cluster management (reuse a worker or add a VM), recover cluster state, show Kubernetes can schedule and manage Pods again, verify DB and transaction integrity.

Failures may be injected at unpredictable times; no manual edits of balances or decisions are allowed.

---

6. Correctness Rules

Same rules as Assignments 1 and 2: one authoritative shard per account, at most one writable DB per shard, durable acknowledged transactions, no duplicate effects from retries, no permanently partial cross-shard transfers, no success reported for unknown outcomes, no manual balance repair. Unavailability is preferred over incorrectness.

7. Deliverables

- **GitHub:** complete source code, Kubernetes manifests, scripts and documentation.
- **Docker Hub:** application images and the image references used by the deployment.

Technical report (max 15 A4 pages) must explain, with details:

- Architecture and Kubernetes deployment: diagram, resource types (Deployment / StatefulSet / Service / Ingress or NodePort), storage, and how shard-aware routing survives Pod changes.
- Shard distribution and service communication: how requests flow from client to gateway, shard and coordinator.
- Database replication and persistence: replication mechanism, failover procedure, and how duplicate or inconsistent effects are avoided.
- Failure handling and transaction recovery: Pod and DB failure behavior, coordinator recovery, and how uncertain transactions are resolved and queried.
- Autoscaling and rolling updates: requests/limits,scaling triggers, performance before/during/after scaling, update and rollback procedure.
- Security and observability: the enforced restriction and the logs/metrics used.
- Test methodology and results: the 7 experiments, workloads used, and DB-state evidence.
- Limitations and observed behavior (plus multi-VM architecture, failure scenarios and recovery results for bonuses).

Demo video (max 20 minutes) should show, in order:

- The deployed system on Minikube: Pods, Services, StatefulSets, PVCs and the external entry point.
- Normal banking operations (balance, deposit, withdraw, same-shard and cross-shard transfers) with shard-aware routing.
- Database replication to the standby.
- Failures and recovery: application Pod kill, primary DB failure, coordinator interruption, with the DB/transaction state checked afterwards.
- Autoscaling driven by a real load, with performance and correctness checked.
- Rolling update under load and a rollback, with correctness checked.
- Repeated client requests producing no duplicate effects, and the enforced security restriction.
- **Bonus I:**
    - Show the three-VM cluster with Pods spread across both workers and replication working between them.
    - Under concurrent load, interrupt one worker. Show services and databases recovering on the healthy worker, with balances intact and never two writable databases.
- **Bonus II:**
    - Worker loss: destroy a worker VM and its disks mid-transaction. Show recovery from surviving resources, no lost acknowledged transactions, and database redundancy rebuilt.
    - Control plane loss: destroy the control plane VM. Restore cluster management without it, show Kubernetes managing Pods again, and verify database integrity.